

Facilitator answer key — Holocron capstone worked solution. Facilitator-only — lives in the facilitator folder of every repo; do not hand to participants. Part of the internally-consistent set in this folder (see `README.md`).

TMD-light — Holocron String Management (release-1 slice)

Scope: source-string lifecycle + governed delivery (CS1–CS7, CS11–12, CS15). Behavior over implementation; the fallback path carries the named invariant.

1. Data model

PostgreSQL is the system of record. Keys and versions are **immutable** — an edit writes a new version row, never mutates an old one. All timestamps `timestampz` . Every mutating op also writes an `AuditEntry` (see below).

Entity	PK	Key FKs
Product	product_id	—
SourceString	string_id	product_id → Prod
SourceStringVersion	src_version_id	string_id → SourceString
TranslationVariant	variant_id	string_id → SourceString
TranslationVersion	tr_version_id	variant_id → TranslationVariant; src_version_ref — SourceStringVersion
ReviewRequest	review_id	src_version_id or tr_version_id; reviewer_id / pool → ReviewerPool
PublicationSelection	selection_id	variant_id → TranslationVariant
AuditEntry	audit_id	actor_id → User; polymorphic entity_ref

Storage trade-off (named): translation state is **denormalized** onto `TranslationVariant` (`translation_state` , `published_selection_id`) rather than derived from the version rows at read time. Delivery cannot afford to recompute "which version is live for this locale" per request, so the write path

maintains it. Cost: the publish/outdate transitions must keep the denormalized pointer honest — enforced in one transaction with the version insert.

Key indexes (3):

1. `idx_delivery` on `SourceString` (`product_id`, `namespace_key`) WHERE `lifecycle_status='Published'` — the namespace-prefix lookup that backs the delivery GET; partial index keeps it to live rows only.
2. `idx_variant_locale` on `TranslationVariant` (`string_id`, `locale`) — one hop from a resolved source string to its per-locale publication selection.
3. `idx_audit_entity_time` on `AuditEntry` (`entity_ref`, `occurred_at` DESC) — the reverse-chronological audit/version view (CS5).

2. Cloud topology

Layer	Azure service	Stance
Compute	AKS	management app + delivery API as separate deployments/HPAs so a delivery spike can't starve management
System of record	Azure Database for PostgreSQL (Flexible Server)	zone-redundant HA primary + read replica
Delivery read cache	Azure Cache for Redis	namespace payloads keyed <code>product:namespace:locale</code> ; publish invalidates
Delivery edge	Application Gateway + Azure API Management	TLS, per-consumer rate limit (50 req/s, burst 100), bearer-token validation
Identity	Microsoft Entra ID	role/scope claims for management; app-registration bearer tokens for consuming apps
Secrets	Azure Key Vault	DB creds, Redis keys, signing config
Event stream	Event Hubs → Blob Storage	every AuditEntry streamed, then archived immutably (WORM container)
Observability	Azure Monitor	metrics, logs, alerts (\$5)

Region / AZ: single primary region, **zone-redundant** across 3 AZs (PostgreSQL zone-redundant HA, AKS node pools spread across zones, zone-redundant Redis). No active-active multi-region for release 1 — the 99.9% delivery SLO is met within one region; a second region is a documented release-2 option, not a release-1 goal.

ROM cost sense: ~\$4–6k/month for release-1 scale (small AKS pool, a mid-tier zone-redundant Postgres, Standard-tier Redis, APIM Standard, Event Hubs Basic + Blob). Delivery traffic — not

string volume (~500k strings is small) — drives cost, and Redis absorbs the read fan-out so Postgres stays modestly sized.

3. API contract

Method	Path
GET	/v1/products/{productId}/namespaces/{namespace}locale=...
POST	/v1/products/{productId}/namespaces/{namespace}
POST	.../strings/{key}/versions
POST	.../strings/{key}/review-requests
PATCH	.../review-requests/{id} { "state":"complete" }
POST	.../strings/{key}/publish

Delivery GET — full contract. Returns only **Published** content for the namespace prefix, resolved for the requested locale:

```
200 OK
[
  { "key": "orders.checkout.submit", "value":
    "Submit order", "locale": "ja-JP", "version": 7,
    "fallback": false },
  { "key": "orders.checkout.tax", "value": "Tax",
    "locale": "en-US", "version": 3, "fallback": true }
]
```

- **401** missing/invalid token · **403** token not authorized for product · **404** unknown product/namespace · **429** consumer over 50 req/s (burst 100), with `Retry-After` .
- **fallback: true** is the explicit indicator that en-US source was served because no published translation exists for the locale (CS7-AS-002 / CS7-FR-003).

Error semantics: management mutations are **transactional** — the version insert, the denormalized state update, and the AuditEntry commit together or not at all. Idempotency keys make POSTs safe to retry through a network blip without creating duplicate strings or duplicate open reviews.

4. Sequence

Happy path — publish → cache invalidation → consuming app fetch (with per-key fallback)

Latencies annotate the **delivery read** only; the publish write is asynchronous to the read and bounded separately by the ≤ 60 s propagation SLO.

```

PUBLISH (management app, async to reads)
  Content Owner → AKS mgmt API      : POST publish
  (HTTPS)          [transaction]
  AKS → PostgreSQL                  : insert
  SourceStringVersion,
                                  set
  lifecycle=Published,
                                  write
  AuditEntry                      (~15 ms)
  AKS → Redis                      : DEL
  product:ns:* (invalidate) (~3 ms)
  AKS → Event Hubs                  : emit audit
  event — → Blob (archive, async)
  — propagation budget: cache repopulates on next
  read, well within ≤60s —

DELIVERY FETCH (consuming app, p95 ≤ 200ms)
  App → App Gateway/APIM            : GET
  /v1/products/{id}/namespaces/{ns}?locale=ja-JP
                                  TLS + validate
  bearer + rate check (~8 ms)
  APIM → AKS delivery API           : forward (HTTP,
  in-cluster)                      (~2 ms)
  AKS → Redis                      : GET
  product:ns:ja-JP                  (~3 ms) ← MISS
  (just invalidated)
  AKS → PostgreSQL                  : idx_delivery
  prefix scan +
                                  per-locale
  PublicationSelection              (~35 ms)
  AKS                              : resolve
  locale; key with no ja-JP
                                  published
  translation → en-US,
                                  fallback=true
  (~4 ms)
  AKS → Redis                      : SET
  product:ns:ja-JP (repopulate)    (~3 ms)
  AKS → App                        : 200 JSON array
  (~5 ms)
  — total ≈ 60 ms cold (cache miss); warm hit ≈ 20
  ms — both inside p95 ≤ 200ms —

```

Trace to acceptance criteria: returns only Published (CS7-AS-001 / CS7-FR-007); per-locale resolution with explicit `fallback` indicator (CS7-AS-002 / CS7-FR-003); publish makes content available to consuming apps (CS3-AS-005); propagation within ≤60s SLO.

Sad path — consumer over rate limit (429)

```
App → App Gateway/APIM : GET ...?locale=ja-JP
(51st req this second)
APIM                      : per-consumer counter >
50 req/s (burst 100 exhausted)
APIM → App                : 429 Too Many Requests +
Retry-After: 1
                          (request never reaches
AKS, Redis, or Postgres)
```

Named invariant: *the rate limiter is enforced at the edge (APIM), before any Holocron read path* — a misbehaving consumer cannot degrade delivery latency or availability for well-behaved consumers, and no partial/uncached read is ever produced by a throttled request. Ties to the 50 req/s (burst 100) SLO and the 99.9% delivery availability target.

5. Performance baseline + monitoring

Baselines (3): delivery fetch **p95 ≤ 200ms / p99 ≤ 500ms**;
delivery availability **99.9%** (management app 99.5%);
publish→delivery propagation **≤ 60s** for normal strings, **< 5s** for
critical/legal -flagged strings (synchronous invalidation, per
SEP §3).

Monitoring plan — one alert per SLO (Azure Monitor):

SLO	Signal (source)	Alert threshold
Delivery latency	APIM/AKS request-duration histogram	p95 > 200ms or p99 > 500ms over 5 min
Delivery availability	APIM 5xx / total, synthetic probe on GET	success rate < 99.9% over rolling 30 min
Management availability	AKS mgmt-API health probe	success rate < 99.5% over 30 min
Propagation	timestamp delta: publish AuditEntry (Event Hubs) → first cache-repopulated read	normal p95 delta > 60s; critical/legal delta > 5s (tighter alert)
Rate limiting	APIM 429 count per consumer	sustained 429s → notify (possible misconfig, not an incident)

Leading indicator (measurable within 7 days): Redis cache-hit ratio on the delivery path. A falling hit ratio predicts rising p95 *before* the latency alert fires — it's the early warning that invalidation is too aggressive or a hot namespace is thrashing. Dashboard it alongside p95 so the correlation is visible day-one.

Additional dashboards: audit-lag (Event Hubs → Blob archive delay, confirms immutable trail is keeping up); DB replica lag; fallback rate per locale (a spike means translations went Outdated after a source republish — a content signal, not an outage).

AI-summary validation note: where an AI-generated summary annotates a dashboard (e.g., "delivery healthy; one namespace trending hot"), it is **advisory only and must be validated against the raw Azure Monitor metrics** before any operator action. The AI summary never gates an alert or suppresses a threshold breach — the numeric SLO signals above are authoritative. Treat the summary as a reading aid, never as the source of truth.

Model answer, ~1.5 pages. Internally consistent with `tcd-light.md` (same Azure stack, same SLOs) and `prd-light.md` (same CS1–CS7 / CS11–12 / CS15 slice). Every SLO in §5 has

an alert; every delivery acceptance criterion in §4 traces to the PRD.